RESEARCH PROJECT PRESENTATION

Slide 1: Agenda

Slide 2: Presentation of serverless

- Serverless ≠ no servers, you just stop thinking about them
- VMs: always on, pay 24/7, manual scaling
- Containers/K8s: better, but you still manage orchestration
- Serverless: scales to zero, pay-per-execution, event-driven, zero infra to manage

Slide 3: Why Serverless

- **Cost**: only pay when code runs, 10-90% cheaper for bursty workloads
- **Scale**: 0 to 10,000 concurrent executions automatically (AWS Lambda)
- **Speed**: 50-75% faster deployment cycles reported by teams
- **Ops**: no patching, no capacity planning, provider handles everything
- **Event-driven**: HTTP, queues, databases, schedules, IoT...
- **HA**: multi-AZ high availability included by default

Slide 4: Framework

- Open-source CLI that reads your `serverless.yml` and deploys all the infra
- Flow: YAML → `sls deploy` → CloudFormation/ARM → live function

4 core concepts:

- **Functions**: units of business logic = one cloud function
- **Events**: what triggers the function (HTTP, S3, SQS, cron...)
- **Resources**: DBs, queues, buckets declared alongside code
- **Plugins**: 1000+ extensions (offline, TypeScript, Datadog...)

Slide 5: yaml file

- Everything in a single YAML file
- `service`: project name, prefix for all created resources
- `provider`: target cloud, runtime, region, stage, IAM permissions

- `functions`: each entry = one Lambda, with its handler and timeout
- `events`: what triggers the function (here a GET `/users/{id}`)
- `resources`: CloudFormation infra (e.g. DynamoDB table) deployed with the code

Slide 6: Multi cloud

- Switching cloud = changing one line in the YAML (`provider: aws` → `azure`)
- **AWS**: Lambda, API Gateway/S3/SQS/EventBridge triggers, 15min timeout, 10k concurrent
- **Azure**: Azure Functions, HTTP/Service Bus/Blob/Event Grid triggers
- **GCP**: Cloud Functions or Cloud Run, Pub/Sub/Firestore/HTTP triggers
- Same CLI commands everywhere: `sls deploy / invoke / logs / remove`

Slide 7-?: Demo

Olle is going to tell his part

Slide ?+1: Community & adoption

- Founded in 2015, 9+ years of maturity
- 59K GitHub stars, 1M+ downloads/week, 1000+ plugins
- Strengths: focus on code, multi-cloud, built-in IaC, large community
- Limitations: cold starts, 15min max on Lambda, local debug requires a plugin, YAML tied to provider

Slide ?+2: Comparaison

- **AWS SAM**: AWS only, native but limited to one cloud
- **Terraform**: very powerful but steep learning curve, not FaaS-first
- **Pulumi**: code-first IaC, good but more complex for functions
- **SST**: great for full-stack AWS apps, but AWS only
- **Serverless Framework**: best choice if you want multi-cloud FaaS quickly without deep IaC expertise

Slide ?+3: Ressources

- **serverless.com/framework/docs**: full YAML, CLI and plugins reference
- **github.com/serverless/examples**: 50+ real-world templates (REST, GraphQL, ML, auth...)
- **serverless-offline**: emulates API Gateway + Lambda locally, essential for dev
- **Serverless Guru**: advanced articles on CI/CD, multi-cloud, performance
- Udemy for practical courses with hands-on labs

Slide ?+4: Takeaway

- Serverless = paradigm shift: event-driven, zero infra, pay-per-use
- The framework unifies AWS, Azure, GCP under one YAML + one CLI
- From zero to a live HTTP endpoint in under 5 minutes

- 59K stars, used by startups and Fortune 500s alike
- Know the trade-offs: cold starts, 15min limit, stateless, vendor-specific YAML